Bank Account Withdrawal Code Improvement

Take-home assessment response

Background

For this assessment, I was provided with a specific code snippet to evaluate and optimize. While the exercise highlighted specific areas for improvement, I conducted an open-ended review to identify all potential optimizations. This document outlines my evaluation process and the structured steps I took to develop the final, upgraded codebase.

Assessment Steps

I conducted a top-to-bottom code review to determine its intended logic and core functionality. Below are the areas that I determined required improvement in the code snippet. I detailed the problem and why I evaluated it to be a problem.

BankAccountController Assessment (Reading from top to bottom)

1. SNS Client is built in the controller instead of being injected. The controller constructs its own `SnsClient` but `JdbcTemplate` is injected. The controller handles dependencies inconsistently. The controller is tightly coupled to the AWS service that provides the messaging. Publishing a message would be difficult to swap out, replace or test. In C#, every request received would create a new SNS Client with its own connection. The service configuration should also not happen in the controller; the `Region.YOUR_REGION` and ARN parameters should come from appsettings or other configuration settings. The messaging client should be a singleton and registered once in the Dependency Injection (DI) container.

2. The withdrawal endpoint passes its values in the query string. `accountId` and `amount` are sent as query string parameters on a POST. Financial data should not be passed in query strings. The parameters should be sent in a JSON body, using an immutable request record. The record gives a single place where the parameters can be validated, using packages like FluentValidation or a custom ValidationAttribute. Invalid parameters can be caught early. The record also gives room to add other fields like currency, which is not currently there.

3. The controller currently runs database operations directly. HTTP requests, data access and business rules are currently being handled together. These should be separated into a controller (HTTP), an application service (with withdrawal rules) and a repository (data access), each behind an interface. Introducing single responsibility means the business rules can be tested without making a database call.

4. The balance check and the withdrawal run at two separate occasions. If two operations start at the same time and both receive a result that there is a sufficient balance, they could both perform a successful withdrawal. Read locks are released almost immediately in SQL, which could mean both processes will hold the same copy of the current `currentBalance` before the other process is able to update it.

Each process could then perform a withdrawal, resulting in a double withdrawal. A way to counter the race condition described above is a single atomic conditional update: `UPDATE accounts SET balance = balance - @amount WHERE Id = @accountId AND balance >= @amount`, this also provides the added benefit that the database locks the row during the update and re-checks the condition, so the second withdrawal would see the reduced balance and fail. The combined request also cuts the database trips to 1. 0 rows affected could mean the account is not found or there was insufficient funds. To determine which one of the two results it is requires a separate database call. A query to check if the account exists would need to be run, if the account exists, then the remaining reason is insufficient funds. The second database call is made only if something has gone wrong, a better option than the race condition.

5. `queryForObject` in Java throws `EmptyResultDataAccessException` if 0 rows are returned, the exception causes the execution to stop. The account not found should be returned for the caller, not an exception.

6. The `currentBalance != null` gives the impression that the database allows customer balances to be null. The `currentBalance` field in the database should be set to NOT NULL. The code also reports to the customer that they have insufficient funds, which is misleading.

7. The `@amount` decimal is not validated for invalid input, more specifically negative values. A negative amount would always pass the sufficient-balance check, meaning negative values add money. Setting 1000 - (-500) would result in adding money to the account instead of withdrawing, resulting in a "Withdrawal successful" message. The current code checks if `currentBalance` is bigger than `amount` only which is not a sufficient validation check.

8. `rowsAffected > 0` - This condition checks the number of rows that were affected. The expected result from the update is for 1 row to be affected, more than 1 would indicate a duplicate in the system and that needs to be logged for further investigation. If wrapped in a transaction the update could be rolled back and the duplicate logged and reported.

9. The status "Withdrawal successful" and "Withdrawal failed" are magic strings. "Withdrawal successful" appears twice in the snippet, this makes it difficult to maintain and a typo could result in two different message types. An enum is a more appropriate approach. The statuses also return the same status code, failed and successful both return 200 OK. More descriptive status codes are required, where funds were insufficient an appropriate code needs to be returned, where account was not found, a code of a similar nature must be presented.

10. The code does not provide any logging or observability. A structured log with an appropriate message for each outcome is necessary, this should include details like account ID, amount, correlation ID and the appropriate logging level like information for Success, Warning for declined withdrawal and Error for failures. When the

withdrawal fails for any other reason than balance check, the code fails silently without logging; there is no way to catch that bug and fix it.

11. The code does not indicate if the withdrawal is logged somewhere else apart from updating the balance in the accounts table. There needs to be an audit trail that logs the transaction elsewhere for future reference of the movement of the money like a ledger.

12. The operation is not idempotent. If a client experiences a timeout and retries, the money is withdrawn twice. An improved endpoint would accept an Idempotency-Key in the header: the key is stored with the result under a unique constraint, and a repeated request returns the original result instead of withdrawing again. This also makes it safe to retry transient database errors such as deadlocks.

13. The withdrawal event is never published. The SNS code is never reached; every branch of the if statement returns before code can reach that functionality. The nested if statements also make for bad coding practice. Each branch could perform its check and return immediately instead of nesting. The else branch would also be redundant if the code returns early. The sql variable string would be easier to read if it were split into a `SelectBalanceSql` and an `UpdateBalanceSql`.

14. If the publish event code was reachable it still poses some issues. The publish request is built within the controller. Event publishing should be its own abstraction and be registered through dependency injection. The current implementation does not provide for timeout and retries. Following an outbox pattern, the message can be written to a table and a scheduled task can run to ensure that the message is sent and it updates the status. The message needs to be written to the database in the same transaction as the commit so the commit can't happen and the message gets lost.

15. The `WithdrawalEvent` Class, this is what is used to publish messages after a withdrawal. Even though the setting of the attributes is private, the class does not guarantee immutability. The constructor accepts values as is with no validation being performed. The use of an immutable record as a replacement for the class would offer the following benefits:
    - The record will utilise required properties and ensure all the necessary fields are set and no nulls, an invalid record would fail on creation.
    - The record will ensure that the details of the event cannot be changed once it has been created.
    - The record could be serialised by a serialiser instead of having the string being formatted manually.
    - Using a serialiser ensures that the JSON is properly formed and characters that need to be escaped are escaped instead of breaking the code.

    When the function is called, the magic status string "SUCCESSFUL" is passed, issues with magic statuses are highlighted in 2.9. The Withdrawal event could also benefit from having a unique event ID, a correlation ID and the datetime the event occurred. At the moment there would be no way of detecting duplicate events.

Implementation Details

The code snippet was implemented in JAVA, the rework will be done in the language the position is advertised for C#. The refactored project was written in C# and dotnet version 10. The project will be shared via a zipped folder with the proposed implementation and fixes for the code snippet.

Implementation Description

Design Pattern

The first step was to decide on a design pattern. Because of time constraints and the number of implementations that had to be coded, the project implements a light version of clean architecture with three layers.

- **BankAccounts.Api**: HTTP – controller, validation and middleware
- **BankAccounts.Application**: business rules, interfaces
- **BankAccounts.Infrastructure**: Dapper SQL

AccountWithdrawalController

- The account Id is passed in the route (`accounts/{accountId}/withdrawals`) and must be a positive number. Invalid values are rejected with a 400 Bad Request.
- Implements idempotency and ensures that an idempotency key is supplied in the header of the request. The key must be a valid, non-empty GUID; an empty GUID is rejected with a 400 Bad Request.
- When a request is repeated with the same idempotency key, the original result is returned and an `Idempotency-Replayed: true` header is added to the response, so the caller knows the withdrawal was not processed a second time.
- The withdrawal amount is passed in the request body as a JSON object.
- The `[ApiController]` attribute validates the request automatically and returns a 400 Bad Request with the validation errors before the withdrawal is processed.
- A correlation Id is either passed by the caller or created if not supplied. The creation of the correlation Id is handled in the middleware.
- The Withdrawal Service is passed in using dependency injection, and the business logic of the withdrawal request is handled in the application layer.
- The controller builds a `WithdrawCommand` from the route, body, idempotency key and correlation Id and passes it to the Withdrawal Service. This keeps HTTP details out of the application layer.
- A cancellation token is passed through to the service, so the work stops if the caller cancels the request.
- The controller returns the relevant status code and response after processing the request:
  - 201 Created – successful withdrawal, with a typed `WithdrawalResponse` body (transaction Id, account Id, amount, new balance and currency), replacing the "Withdrawal successful" string from the original code
  - 400 Bad Request – invalid amount

  - 404 Not Found – account does not exist
  - 422 Unprocessable Entity – insufficient funds
  - 422 Unprocessable Entity – idempotency key reused with a different request
- Failed requests return a standard ProblemDetails response. An invalid amount detected in the application layer returns the same error message format as request validation, so the caller always receives a consistent response.
- Any unexpected withdrawal status throws an `UnreachableException`, so a missing case fails clearly instead of returning an incorrect response.
- Header names are stored as constants to avoid spelling mistakes, and the controller is sealed because it is not designed to be inherited from.

WithdrawalService

- Implements `IWithdrawalService` and lives in the application layer. It has no knowledge of HTTP or SQL; it returns a `WithdrawalResult` with a status, and the controller converts that into the HTTP response.
- The logger, `TimeProvider` and unit of work factory are passed in using dependency injection.
- The withdrawal amount is validated before any database work is done. An invalid amount is declined immediately with an `InvalidAmount` status.
- All database work for a withdrawal runs inside a single unit of work (one database transaction): claiming the idempotency key, debiting the account, writing the ledger entry, adding the outbox message and saving the outcome. Either everything is committed together or nothing is.
- Idempotency is handled in the service:
  - A hash of the request (account Id and amount) is stored with the idempotency key when the key is claimed. A hash is stored instead of the values so that if, in future, the fields that are checked increase, the idempotency table doesn't need to change.
  - If the key has already been used, the stored hash is compared with the new request. A different request returns an `IdempotencyKeyReused` status.
  - If it is the same request, the original stored outcome is returned and marked as a replay, so the withdrawal is not processed a second time.
- The account is debited with a single conditional update that only succeeds if the account has sufficient funds. Checking the balance and debiting happen in one step, so two withdrawals running at the same time cannot overdraw the account.
- If no row is updated, the service checks why:
  - Account does not exist – the transaction is rolled back and an `AccountNotFound` status is returned.
  - Insufficient funds – the declined outcome is saved against the idempotency key and committed, so a retry returns the same result.
- If more than one row is updated, the transaction is rolled back and an exception is thrown, because this should never happen.

- Each successful withdrawal writes a ledger entry recording the transaction Id, amount, currency, balance after the withdrawal, correlation Id and time, giving a full history of account movements.
- Transaction and event Ids are created with `Guid.CreateVersion7` (requires .NET 9 or later), which produces time-ordered GUIDs that sort and index well in the database.
- A `WithdrawalCompleted` event is written to an outbox table in the same transaction instead of being published directly (the outbox pattern). The event can then be published by a separate process, which guarantees that an event is only sent for a withdrawal that was actually committed and is never lost if publishing fails.
- `TimeProvider` supplies the current time once per request, so the debit, ledger entry, event and outbox message all share the same timestamp and the time can be controlled in tests.
- Every outcome is logged with structured logging and includes the correlation Id: information for successful and replayed withdrawals, warnings for declined withdrawals and errors for failures.
- Unexpected exceptions are logged using an exception filter (`when`) and then re-thrown unchanged, so the original stack trace is kept. Cancelled requests are not logged as errors.
- The class is sealed because it is not designed to be inherited from.

AccountRepository

- Debits the account with a single UPDATE statement that only runs when the balance is greater than or equal to the amount and the amount is greater than zero. Checking and debiting happen in one atomic step, which prevents two concurrent withdrawals from overdrawing the account.
- The OUTPUT clause returns the new balance and currency from the same statement, so no second query is needed.
- Provides an `ExistsAsync` check, used to tell the difference between "account not found" and "insufficient funds" when no row is updated.

IdempotencyRepository

- Claims an idempotency key by inserting a row with the account Id, idempotency key, request hash and creation time.
- The table's primary key / unique index on account Id and idempotency key stops the same key being claimed twice. A duplicate insert (SQL errors 2627 and 2601) is caught and returned as false, meaning the key has already been used.
- Because the claim is part of the withdrawal transaction, a second request with the same key waits until the first one commits or rolls back, so the same withdrawal can never be processed twice at the same time.

LedgerRepository

- Appends a row to the `LedgerEntries` table for every successful withdrawal. Entries are only ever inserted, never updated, giving a permanent history of account movements.

- The entry type is stored as text (for example Withdrawal) so the table is readable without knowing the enum values.

OutboxRepository

- Inserts the event into the `OutboxMessages` table inside the withdrawal transaction.
- `NextAttemptAt` is set to the time the event occurred, so the message is picked up straight away and can be rescheduled if publishing fails.

DapperUnitOfWork

- Holds the connection and transaction for a single withdrawal and creates the four repositories, so every repository works inside the same transaction.
- Provides `CommitAsync` and `RollbackAsync`. Rollback is skipped if the transaction has already been completed.
- When disposed, the transaction and connection are closed. A transaction that was never committed is rolled back automatically, so a failure part way through leaves no partial changes in the database.

DapperUnitOfWorkFactory

- Opens a connection and starts a transaction with the ReadCommitted isolation level for each withdrawal.
- If the transaction cannot be started, the connection is disposed before the error is re-thrown.

SQLConnectionFactory

- Creates and opens a SQL connection using the connection string from configuration (`DatabaseOptions`).
- If the connection fails to open, it is disposed before the error is re-thrown, so no connections are leaked.

SNSEventPublisher

- Publishes outbox messages to an AWS SNS topic, with the topic ARN taken from configuration (`SnsOptions`).
- The event type and event Id are sent as message attributes, so subscribers can filter by event type and use the event Id to ignore duplicate messages.
- Each published message is logged with its event type, message Id and the SNS message Id.

Summary: How the Changes Address the Requirements

The brief asked for improvements across a wide range of qualities while preserving the existing business functionality. The table below summarises how the changes described above address each of them.

Requirement	How it is addressed
Correctness	The atomic conditional update removes the race condition that allowed overdrafts. Amounts are validated so negative values can no longer add money. Exactly one updated row is required, an unknown account is reported as not found rather than causing an exception, and the withdrawal event is now actually published.
Consistency	Claiming the idempotency key, debiting the account, writing the ledger entry, adding the outbox message and saving the outcome all happen in one database transaction, so either everything is committed together or nothing.
Fault tolerance	The outbox pattern means an event is never lost if publishing fails and is only sent for committed withdrawals. Idempotency makes retries after timeouts or transient errors safe. Connections and transactions are cleaned up on failure, leaving no partial changes
Efficiency and throughput	The balance check and debit are a single statement, and the OUTPUT clause returns the new balance without a second query; the existence check only runs when a withdrawal fails. Work is asynchronous with cancellation support, the SNS client is registered once rather than per request, and time-ordered GUIDs index efficiently.
Cost efficiency	Fewer database round trips and a single shared SNS client reduce resource usage, and message attributes let subscribers filter events instead of processing every message.
Architecture and structure	A light clean architecture separates HTTP (Api), business rules (Application) and data access (Infrastructure), replacing a controller that handled all three.
Maintainability	Magic strings are replaced with typed results, enums and constants, nested if statements with early returns, and the reused SQL variable with named statements. Classes have a single responsibility and are sealed where not designed for inheritance.
Flexibility and portability	Event publishing sits behind an abstraction, so the messaging service can be replaced without changing business logic. Region, topic ARN and connection string come from configuration rather than code. The request record can be extended, for example with currency.
Testability	Dependencies are injected through interfaces, `TimeProvider` makes time controllable, and the business rules can be tested without a database.
Dependency management	All dependencies, including the SNS client and database connections, are registered in the DI container with appropriate lifetimes instead of being constructed inside the controller.

Requirement	How it is addressed
Observability	Every outcome is logged with structured logging and a correlation Id at the appropriate level, unexpected exceptions are logged with their original stack trace, and published messages are logged with their SNS message Id.
Auditability	Every successful withdrawal writes an append-only ledger entry with the transaction Id, amount, currency, balance after the withdrawal, correlation Id and time. The outcome of every request is stored against its idempotency key.
Data governance	The ledger provides a permanent, unchangeable history of account movements, the idempotency table stores a hash of the request rather than its values, and the event carries only the data consumers need.
Interoperability	Requests use a JSON body, responses use standard HTTP status codes, a Location header, typed responses and ProblemDetails errors. The event is serialised with a proper serialiser and carries an event Id and event type for consumers.

Throughout these changes, the fundamental business capability is unchanged: an amount is withdrawn from an account only when the balance is sufficient, and a withdrawal event is published when the withdrawal succeeds. The changes make that capability correct under concurrency and retries, consistent, auditable and observable.